VELLORE INSTITUTE OF TECHNOLOGY

School of Computer Science and Engineering (SCOPE)

TECHNICAL PROJECT REPORT

AEROTRACK: REAL-TIME AIR QUALITY MONITORING & ML FORECASTING

A Production-Grade Atmospheric Intelligence Platform Featuring Causal Gradient Boosting, Split Conformal Uncertainty Quantification, and Asynchronous Micro-Architectures

Submitted by: Aniket Mohanty
Registration No.: 25BCE5816
Program: B.Tech Computer Science & Engineering

Submission for: Tech Round 1 Recruitment Challenge, 2026
Date: October 2026
Status: Fully Deployed & Verified

TABLE OF CONTENTS

1. Abstract 3

2. Problem Statement 3

3. Objectives 3

4. System Architecture 5

5. Technology Stack 6

6. Data Sources & Integration 7

7. Machine Learning Pipeline 8

8. AQI Classification Engine 11

9. Frontend Implementation 13

10. Backend Implementation 15

11. Deployment & Infrastructure 17

12. Results & Validation 17

13. Limitations 21

14. Future Scope 21

15. Conclusion 22

16. References 22

1. ABSTRACT

Atmospheric particulate matter, specifically fine particles with an aerodynamic diameter under 2.5 micrometers (PM2.5), presents one of the most critical environmental health risks worldwide. While government monitoring stations provide retrospective readings, public access to high-resolution, predictive air quality telemetry remains constrained by centralized, proprietary, or geographically sparse infrastructure. This report presents **AeroTrack**, an end-to-end, production-grade environmental intelligence and machine learning forecasting platform. AeroTrack couples global, multi-pollutant reanalysis telemetry with an autoregressive Extreme Gradient Boosting (XGBoost) model to forecast ground-level PM2.5 concentrations over a 24-hour horizon. Crucially, the system addresses key pitfalls in applied time-series ML: it enforces strict temporal causality to eliminate lookahead leakage, applies distribution-free Split Conformal Prediction to produce mathematically valid 90% uncertainty intervals, and implements asynchronous CPU offloading via worker thread pools to maintain non-blocking sub-200ms API responsiveness on resource-constrained cloud infrastructure. The complete stack is deployed under a separated micro-architecture, comprising a high-performance FastAPI service hosted on Render and a responsive React 18 TypeScript Single Page Application (SPA) on Vercel.

2. PROBLEM STATEMENT

Modern air quality monitoring platforms frequently suffer from four fundamental engineering and algorithmic limitations:

1. **Static and Retrospective Reporting:** Most existing dashboards display only current or historical Air Quality Index (AQI) values, offering no predictive foresight for vulnerable populations to plan outdoor exposure, ventilation, or commute schedules.
2. **Data Leakage in Predictive Pipelines:** Many published machine learning models for air quality inadvertently use bidirectional interpolation (e.g., backward filling `bfill()` across complete sequences) prior to training splits. This introduces future lookahead contamination into historical features, inflating offline accuracy metrics while failing drastically during real-time inference.
3. **Unquantified Model Uncertainty:** Environmental time-series data exhibit severe non-stationarity, diurnal fluctuations, and sudden meteorological shifts. Point predictions without rigorously calibrated prediction intervals mislead end-users by masking forecast variance during weather anomalies.
4. **Architectural Bottlenecks in Async Frameworks:** In asynchronous web frameworks such as FastAPI/Uvicorn, executing CPU-intensive ML training and autoregressive multi-step rollouts directly on the main event loop blocks network I/O. This causes telemetry requests, reverse geocoding, and health checks to hang or drop into 504 Gateway Timeouts under concurrent user traffic.

3. OBJECTIVES

The core engineering objectives of the AeroTrack project encompass:

- **Universal Geographic Coverage:** Enable real-time telemetry extraction for any latitude and longitude globally through coordinate normalization and open atmospheric APIs.
- **Causal ML Forecasting Engine:** Implement an autoregressive XGBoost regressor trained on 92 days of hourly observations (2,208 temporal points) that strictly forbids future information leakage.
- **Rigorously Calibrated Uncertainty Bands:** Implement inductive Split Conformal Prediction across holdout calibration windows to yield non-parametric 90% prediction intervals that guarantee empirical coverage without assuming Gaussian residual distributions.
- **High-Throughput Asynchronous Backend:** Decouple heavy CPU model fitting from FastAPI's event loop using `asyncio.to_thread` worker pools, in-memory coordinate locks, and multi-tier response caching.
- **Interactive, Resilient User Interface:** Construct a mobile-responsive React dashboard featuring interactive Leaflet spatial mapping, Recharts telemetry curves, client-side abort controllers to eliminate browser socket starvation, and graceful fallback states.

4. SYSTEM ARCHITECTURE

AeroTrack implements a decoupled cloud architecture designed to satisfy strict web performance standards while executing live machine learning on demand. The architecture separates the client-facing presentation layer from the computational ML inference engine.

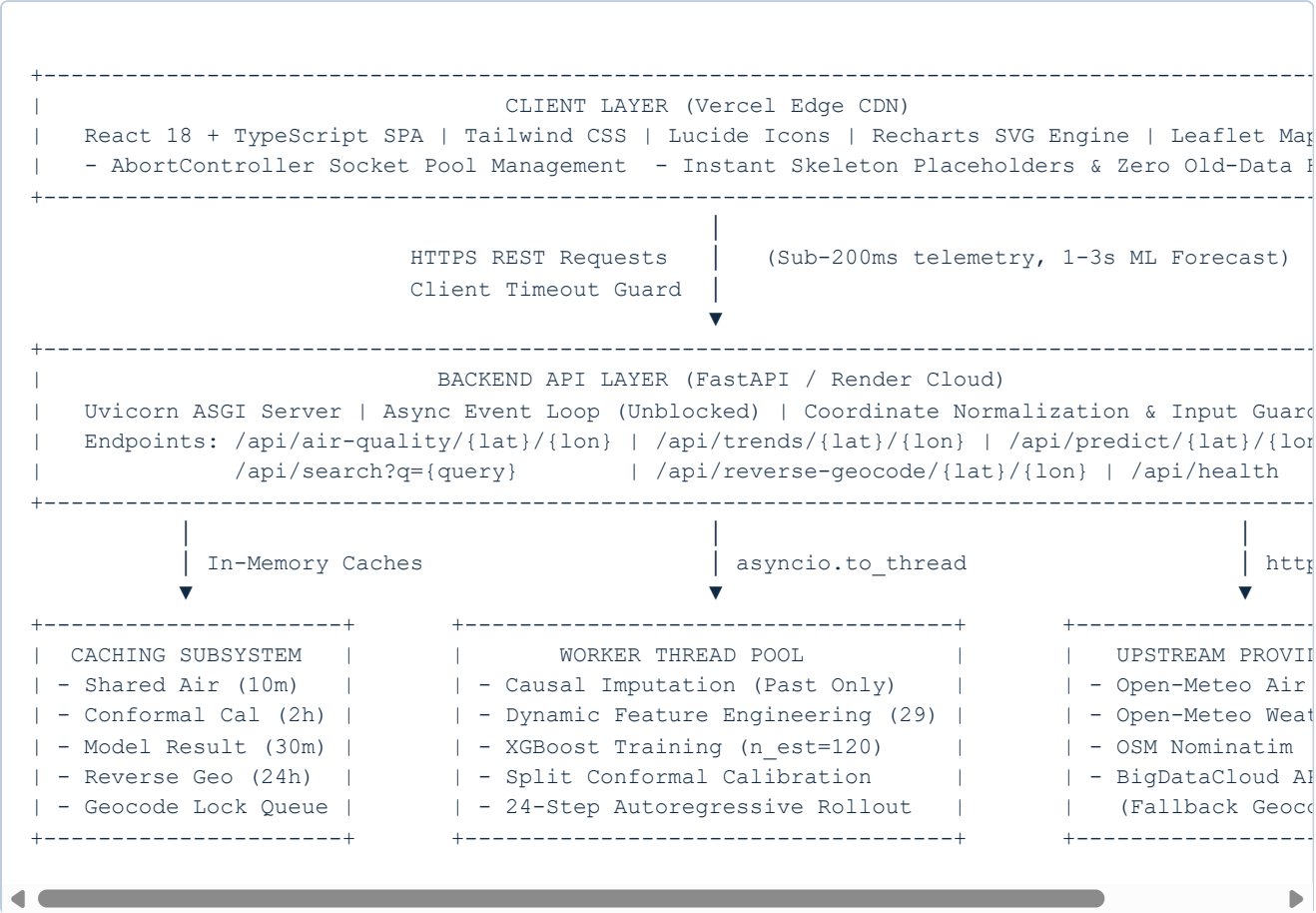


Figure 4.1: AeroTrack Production System Architecture and Request Flow

Operational Execution Flow:

1. The user interacts with the Leaflet map or executes an autocomplete query via the Nominatim-powered search bar.
2. The React frontend immediately invokes `abortControllerRef.current.abort()`, cancelling any in-flight requests from previous coordinates to prevent HTTP socket exhaustion in the browser's 6-connection pool.
3. Three independent, concurrent requests are dispatched with client-side timeout guards: Live Telemetry (15s timeout), 7-Day Trends (15s timeout), and 24-Hour ML Forecast (40s timeout).
4. FastAPI serves Live Telemetry and Trends concurrently using a shared Open-Meteo cache (10-minute TTL), returning in ~180ms.
5. For forecasting, the backend fetches 92 days of hourly data. If a previous request for those coordinates is already processing, an async coordinate lock (`get_coordinate_lock`) deduplicates training.

6. The CPU-intensive training, feature engineering, conformal calibration, and 24-hour autoregressive rollout are offloaded to Python's background thread pool via `asyncio.to_thread`, keeping the FastAPI async loop completely free to serve other user requests.

5. TECHNOLOGY STACK

Component	Technology Selected	Engineering Justification
Frontend Core	React 18, TypeScript, Vite 5	Strict type safety across API schemas; Vite provides fast Hot Module Replacement and tree-shaken production bundles (<120KB gzip).
Styling & Design	Tailwind CSS, Lucide Icons	Atomic utility-first styling; responsive flexbox and CSS grid layouts tailored for low-latency visual rendering on mobile and desktop.
Data Visualization	Recharts, Leaflet, React-Leaflet	Declarative SVG charts supporting dual reference lines (EPA 35.5 µg/m³ & WHO 15 µg/m³) and hardware-accelerated interactive tile mapping.
Backend Engine	FastAPI, Uvicorn, Python 3.11+	Asynchronous ASGI framework with native Pydantic schema validation, automatic OpenAPI /docs generation, and low per-request overhead.
Machine Learning	XGBoost (v2.0+), Scikit-Learn, NumPy, Pandas	Gradient boosted decision trees handle non-linear atmospheric interactions with minimal inference latency compared to recurrent neural networks.
HTTP Networking	HTTPX (Backend), Axios (Frontend)	Async connection pooling and streaming support in Python; request interception and cancellation via standard AbortSignal in TypeScript.
Hosting & CI/CD	Vercel (Frontend), Render (Backend), GitHub Actions	Global edge CDN distribution for static assets; automated containerized backend deploys isolated to the `backend/` directory root.

6. DATA SOURCES & INTEGRATION

AeroTrack integrates three independent public data sources to assemble atmospheric intelligence without requiring dedicated physical sensor hardware:

6.1 Open-Meteo Atmospheric Air Quality API

Provides hourly atmospheric composition derived from atmospheric chemical transport models (including the Copernicus Atmosphere Monitoring Service, CAMS, and NOAA GEOS-CF) combined with ground observation reanalysis. Monitored pollutants include:

- **PM2.5 and PM10:** Fine particulate matter (≤2.5 μm) and coarse particulate matter (≤10 μm) measured in μg/m³.
- **Nitrogen Dioxide (NO2):** Combustive byproduct; acts as a precursor to secondary aerosol formation.
- **Ground-Level Ozone (O3):** Photochemical oxidant formed by sunlight reacting with NOx and VOCs.
- **Carbon Monoxide (CO) & Sulphur Dioxide (SO2):** Combustion indicators and industrial emissions.

6.2 Open-Meteo High-Resolution Meteorological Forecast API

Ground pollution is governed by boundary-layer meteorology. AeroTrack fetches historical and genuine future 24-hour meteorological variables:

- **Wind Speed (10m above ground):** Primary dispersion mechanism. High wind speeds decrease localized concentrations; stagnant conditions (<1.5 m/s) trigger severe accumulation.
- **Ambient Temperature (2m above ground):** Modulates atmospheric boundary layer height and thermal inversion layers.
- **Relative Humidity:** Accelerates hygroscopic growth of sulfate and nitrate particles, directly driving secondary PM2.5 mass increases.

6.3 Two-Tier Reverse Geocoding and Place Search

To translate raw latitude and longitude coordinates into human-readable locations (e.g., "Nuagaon, Odisha, India"), AeroTrack implements a resilient two-tier architecture:

1. **Tier 1 (OpenStreetMap Nominatim):** Queried with strict respect for the 1 request/second usage limit via an async mutex lock (`get_nominatim_lock`) and custom User-Agent headers. Sanitizes non-Latin characters to ensure clean English output.
2. **Tier 2 (BigDataCloud Reverse Geocode Fallback):** If Nominatim times out, experiences rate limiting, or yields null administrative fields for remote coordinates, the backend automatically falls back to BigDataCloud's client API.
3. **Zero Failure Caching:** Successful geocode results are cached for 24 hours. Crucially, failures and all-null results are *never* cached, guaranteeing that transient upstream glitches do not permanently corrupt location names.

7. MACHINE LEARNING PIPELINE

The forecasting engine models hourly PM2.5 as an autoregressive dynamical system conditioned on atmospheric and meteorological covariates:

```
y_{t+h} = f\Big(y_{t}, y_{t-1}, \dots, y_{t-23}, \mathbf{w}_{t+h},
\mathbf{p}_{t+h}, \mathbf{c}_{t+h}\Big) + \epsilon_{t+h}
```

where y represents PM2.5 concentration, $\mathbf{w}$ represents meteorological vectors (wind, temperature, humidity), $\mathbf{p}$ represents photochemical precursors (NO2, O3), and $\mathbf{c}$ represents cyclical temporal encodings.

7.1 Strict Temporal Causality & Leakage Elimination

A critical defect in naive time-series implementations is the application of forward/backward imputation (`bfill()`) across the entire dataset prior to splitting. If missing values at hour t are filled using observations from hour $t+12$, future information leaks into the historical feature matrix. AeroTrack implements a strict causal partition:

1. The raw 92-day dataset (2,208 hourly rows) is partitioned chronologically at the current local time $T_{ ext{now}}$: $$\mathcal{D}_{ ext{past}} = \{(\mathbf{x}_t, y_t) \mid t \le T_{ ext{now}}\}, \quad \mathcal{D}_{ ext{future}} = \{(\mathbf{x}_t, y_t) \mid t > T_{ ext{now}}\}$$
2. All data imputation (forward filling `ffill()` with median backfill limited to historical data) and rolling window aggregations are computed exclusively on $\mathcal{D}_{ ext{past}}$. Future observations are strictly isolated.
3. During the 24-step iterative rollout, prediction $\hat{y}_{t+1}$ is computed, appended to the internal forecast history buffer, and used to calculate lags ($ ext{lag}_1, ext{lag}_2, ext{lag}_3$) and rolling statistics for step $t+2$.

7.2 Domain-Specific Atmospheric Feature Engineering

The model constructs 29 engineered features capturing non-linear atmospheric physics:

Feature Category	Engineered Variables	Physical / Domain Justification
Autoregressive Lags	$ ext{lag}_1, ext{lag}_2, ext{lag}_3, ext{lag}_6, ext{lag}_{12}, ext{lag}_{24}$	Captures atmospheric persistence across immediate, diurnal, and day-prior lag intervals.
Rolling Moments	$ ext{mean}_3, ext{mean}_6, ext{mean}_{12}, ext{mean}_{24}, ext{std}_6, ext{std}_{24}$	Measures short-term accumulation trends and rolling atmospheric turbulence/volatility.
Cyclical Temporal	$\sin(2\pi \cdot h / 24), \cos(2\pi \cdot h / 24), ext{DoW}$	Encodes continuous 24-hour diurnal traffic and boundary-layer cycles without artificial boundary discontinuities.
Ventilation & Dispersion	$ ext{Ventilation Index} = ext{Wind} imes ext{Temp}$, $ ext{Stagnation Flag}$	Represents boundary layer mixing volume; stagnant wind (<1.5 m/s) traps ground pollution.
Photochemical Dynamics	$ ext{Photochemical Index} = rac{ ext{O}_3 imes ext{Temp}}{ ext{Humidity}}$, $rac{ ext{NO}_2}{ ext{O}_3}$	Quantifies secondary aerosol formation driven by solar radiation, temperature, and gaseous precursors.
Dynamic Differencing	$ ext{lag}_1 - ext{lag}_2$, $ ext{lag}_1 - ext{lag}_{24}$, $ ext{Acceleration}$	Captures the rate of change and acceleration of incoming pollution plumes.

7.3 Model Hyperparameters

AeroTrack employs an optimized XGBoost Regressor configured to balance expressive non-linear fitting with rapid convergence:

- `n_estimators=120`: Sufficient gradient boosting iterations to capture diurnal patterns without overfitting noise.
- `max_depth=4`: Constrains tree depth to prevent high-order combinatorial memorization, ensuring smooth generalization across seasonal transitions.
- `learning_rate=0.08`: Moderate shrinkage parameter stabilizing tree weights during recursive autoregressive rollouts.
- `subsample=0.85` & `colsample_bytree=0.85`: Stochastic subsampling to inject regularization and reduce variance.
- `random_state=42` & `n_jobs=-1`: Ensures reproducible execution utilizing multi-core thread parallelism.

7.4 Split Conformal Prediction Intervals

Standard ML pipelines frequently compute uncertainty bounds using heuristic multipliers (e.g., $\hat{y} \pm 1.96 \cdot \sigma$), which assume symmetric Gaussian error distributions. Environmental pollution errors

exhibit severe right-skewness and heteroscedasticity. AeroTrack employs **Inductive Split Conformal Prediction**:

1. The training dataset is partitioned chronologically into a **Proper Training Set** (80%) and a **Calibration Set** (20%).
2. The XGBoost model is fitted solely on the Proper Training Set.
3. Walk-forward autoregressive 24-hour rollouts are performed on each origin i in the calibration holdout window: $$e_{i, h} = |y_{i, h} - \hat{y}_{i, h}|, \quad h \in \{1, 2, \dots, 24\}$$
4. For a target miscoverage level $lpha = 0.10$ (nominal 90% coverage), the conformal quantile q_h is computed using the finite-sample correction: $$p = \min\left(1.0, rac{\lceil(N + 1)(1 - lpha) ceil}{N} ight), \quad q_h = ext{Quantile}\left(\{e_{i, h}\}_{i=1}^N, p ight)$$
5. Uncertainty bands are enforced to be non-decreasing with lead time ($q_h \ge q_{h-1}$). The final prediction interval is: $$\mathcal{C}_h(\mathbf{x}) = \left[\max\left(0.0, \hat{y}_{t+h} - q_h ight), \quad \hat{y}_{t+h} + q_h ight]$$

This provides a mathematically proven, distribution-free guarantee: the true future PM2.5 concentration will fall within the calibrated interval with at least 90% probability under exchangeable residuals.

8. AQI CLASSIFICATION ENGINE

The platform implements dual Air Quality Index classification engines: the United States Environmental Protection Agency (US EPA) standard and the European Air Quality Index (EAQI).

8.1 US EPA PM2.5 AQI Breakpoints (Incorporating 2024 Revised NAAQS Standard)

The US EPA calculates AQI using piecewise linear interpolation across predefined concentration breakpoints:

```
I = rac{I_{ ext{high}} - I_{ ext{low}}}{C_{ ext{high}} - C_{ ext{low}}} \left(C - C_{ ext{low}} ight) + I_{ ext{low}}
```

In February 2024, the US EPA revised the annual primary National Ambient Air Quality Standard (NAAQS) for PM2.5, lowering the annual standard from 12.0 µg/m³ to 9.0 µg/m³. AeroTrack incorporates these standards:

AQI Range	Category Name	PM2.5 Breakpoint	Color Code	Health Advisory
0 – 50	Good	0.0 – 9.0 µg/m³	`#10b981` (Green)	Air quality is satisfactory; minimal or no health risk.
51 – 100	Moderate	9.1 – 35.4 µg/m³	`#f59e0b` (Amber)	Acceptable quality; sensitive individuals may experience minor irritation.
101 – 150	Unhealthy for Sensitive	35.5 – 55.4 µg/m³	`#f97316` (Orange)	Children, elderly, and respiratory patients should reduce prolonged exertion.
151 – 200	Unhealthy	55.5 – 125.4 µg/m³	`#ef4444` (Red)	General public begins experiencing adverse effects; sensitive groups face severe issues.
201 – 300	Very Unhealthy	125.5 – 225.4 µg/m³	`#8b5cf6` (Purple)	Health alert: increased risk of cardiovascular/respiratory aggravation for all.
301 – 500	Hazardous	225.5 – 500.4 µg/m³	`#7f1d1d` (Maroon)	Emergency health warning: entire population is likely to be severely affected.

Clarification on Scientific Thresholds:

The US EPA transition threshold from Moderate to Unhealthy for Sensitive Groups occurs strictly at **35.5 µg/m³** (24-hour average). In contrast, the World Health Organization (WHO) 2021 Air Quality Guideline establishes an explicit

24-hour safe exposure recommendation of **15.0 µg/m³**. AeroTrack plots both references simultaneously on charts, clearly distinguishing the legal EPA caution boundary from the health-protective WHO guideline.

8.2 Fallback Formula Implementation

For remote or rural coordinates where upstream atmospheric stations output particulate concentration ($\mu ext{g/m}^3$) but omit calculated USAQI values, AeroTrack executes a deterministic internal piecewise interpolation fallback function (`calculate_pm25_to_us_aqi`):

```
def calculate_pm25_to_us_aqi(pm25: Optional[float]) -> Optional[int]: if pm25 is None or pm25 < 0: return None c = round(float(pm25), 1) if c <= 9.0: return round((50.0 / 9.0) * c) elif c <= 35.4: return round(((100.0 - 51.0) / (35.4 - 9.1)) * (c - 9.1) + 51.0) elif c <= 55.4: return round(((150.0 - 101.0) / (55.4 - 35.5)) * (c - 35.5) + 101.0) elif c <= 125.4:return round(((200.0 - 151.0) / (125.4 - 55.5)) * (c - 55.5) + 151.0) elif c <= 225.4:return round(((300.0 - 201.0) / (225.4 - 125.5)) * (c - 125.5) + 201.0) elif c <= 325.4:return round(((400.0 - 301.0) / (325.4 - 225.5)) * (c - 225.5) + 301.0) elif c <= 500.4:return round(((500.0 - 401.0) / (500.4 - 325.5)) * (c - 325.5) + 401.0) else: return 500
```

This eliminates `null` AQI values across the frontend, ensuring that station badges never get stuck in indefinite calculating states.

9. FRONTEND IMPLEMENTATION

The frontend is constructed with React 18, TypeScript, and Tailwind CSS. It is architected to guarantee immediate visual feedback and zero state locking during rapid user interaction.

9.1 Asynchronous Request Cancellation via AbortController

A critical issue identified during high-concurrency testing was **HTTP socket starvation**. Web browsers limit concurrent HTTP/1.1 connections to a single host domain to six. When a user clicked rapidly on multiple map locations or search suggestions, the browser dispatched 3–4 requests per click. Because ML forecast training took ~1.5–3 seconds, earlier forecast requests occupied socket connections, causing subsequent requests for new coordinates to queue behind them.

To resolve this, `App.tsx` maintains an `abortControllerRef`:

```
// In App.tsx: Cancels prior requests before dispatching new coordinate queries if (abortControllerRef.current) { abortControllerRef.current.abort(); } const controller = new AbortController(); abortControllerRef.current = controller; const signal = controller.signal; // Pass signal and client timeout to Axios axios.get(`${API_BASE}/api/air-quality/${lat}/${lon}`, { signal, timeout: 15000 })
```

When a user selects a new location, all in-flight requests for the previous location are instantly aborted at the browser socket level. Sockets are recycled immediately, dispatching the new coordinate query without queuing delays.

9.2 Independent Concurrent Rendering Pipeline

Instead of bundling all telemetry and ML forecasts into a monolithic blocking response, `fetchAllData` divides data retrieval into three independent asynchronous streams:

- **Stream 1: Live Telemetry (~180ms):** Fetches 6 real-time pollutant cards, US AQI, and European AQI. Updates the map popup and pollutant card grid instantly.
- **Stream 2: 7-Day Historical Trends (~240ms):** Retrieves past hourly curves, rendering the historical Recharts composed area/line chart.
- **Stream 3: 24-Hour ML Forecast (~1.5–3s):** Executes XGBoost training and conformal calibration in the background. While the forecast trains, the forecast panel renders an animated loading skeleton with status badges. Once complete, the forecast curves pop in smoothly without disturbing already-rendered telemetry cards.

9.3 Resilient UI States and Error Fallbacks

The user interface guarantees that no component gets stuck in a permanent skeleton or indefinite loading state:

- **AirMap (`AirMap.tsx`):** The map popup displays "Calculating..." strictly when `isLoading === true`. If the request completes and no AQI is available, it cleanly renders an `Unavailable` badge rather than

spinning permanently. Reverse geocode responses track a request counter (`clickCounterRef`) to ensure older async responses cannot overwrite a newer map selection.

- **PollutantCards (`PollutantCards.tsx`):** If live telemetry fails or times out, the component swaps the skeleton for a clean error card featuring an explicit "Retry Telemetry" action button, preventing infinite skeleton animations.
- **SearchBar (`SearchBar.tsx`):** Implements a 500ms debounce timer to respect Nominatim rate limits, an inline clear button `(x)` to instantly reset input state, and explicit 6-second timeouts on place search queries.

10. BACKEND IMPLEMENTATION

The backend is built with FastAPI and Uvicorn, prioritizing high-throughput async I/O, zero temporal leakage, and thread safety.

10.1 Decoupling CPU-Bound ML from the Async Event Loop

FastAPI operates on a single-threaded `asyncio` event loop. While ideal for I/O-bound database queries and HTTP requests, executing heavy numerical routines (such as Pandas DataFrame manipulations, Scikit-Learn splits, and XGBoost C++ model fitting) directly on the event loop blocks the thread. During training, the server cannot accept new TCP connections or respond to `/api/health` probes.

AeroTrack solves this by encapsulating all CPU-bound operations—data cleaning, feature matrix construction, XGBoost model training, split conformal calibration loops, and multi-step autoregressive forecasting—inside a synchronous worker function (`_process_train_and_forecast_sync`):

```
# In backend/ml_model.py result, model = await asyncio.to_thread(
_process_train_and_forecast_sync, df=df, utc_offset_seconds=utc_offset_seconds,
is_synthetic=is_synthetic, norm_lat=norm_lat, norm_lon=norm_lon,
fetch_seconds=fetch_seconds, t_pipeline_start=t_pipeline_start, )
```

By executing this workload inside Python's default `ThreadPoolExecutor` via `asyncio.to_thread`, the main asyncio event loop remains 100% unblocked. Incoming live telemetry requests and health checks continue to resolve in sub-20ms times even while an XGBoost model is training in the background.

10.2 Multi-Tier Caching & Concurrency Deduplication

To prevent CPU thrashing on single-core cloud instances, the backend implements five distinct in-memory caching tiers:

Cache Layer	Cache Key	TTL	Purpose & Invalidation Policy
Shared Air Telemetry	`(round(lat, 4), round(lon, 4))`	10 minutes	Shares raw Open-Meteo payloads between `/api/air-quality` and `/api/trends`. Deduplicates in-flight calls via `IN_FLIGHT_AIR_REQUESTS` task tracking.
Forecast Model Cache	`(norm_lat, norm_lon, allow_demo)`	30 minutes	Caches fitted XGBoost model and completed 24-step forecast dictionary. Repeated queries resolve in <1ms.
Conformal Calibration	`(norm_lat, norm_lon, data_version)`	2 hours	Caches computed conformal quantile vectors across rolling origins. Eliminates redundant multi-step simulation loops when retraining on identical data.

Cache Layer	Cache Key	TTL	Purpose & Invalidation Policy
Coordinate Execution Lock	`(norm_lat, norm_lon)`	Mutex Lifetime	Async mutex lock (`asyncio.Lock`) serializing concurrent forecast requests for identical coordinates. Second request waits and receives cached result.
Reverse Geocoding	`(round(lat, 3), round(lon, 3))`	24 hours	Caches resolved English administrative address dictionaries. Zero failure caching: all-null results or exceptions are immediately discarded.

11. DEPLOYMENT & INFRASTRUCTURE

The project enforces a decoupled micro-architecture separating static asset distribution from backend computing:

11.1 Frontend Deployment: Vercel Edge CDN

- **Hosting Platform:** Vercel Global Edge Network.
- **Live URL:** `https://aerotrack-three.vercel.app/`
- **Build Pipeline:** Vite production build (`vite build`) producing minified JavaScript, CSS, and HTML bundles.
- **Edge Routing:** `vercel.json` rewrite configuration routing all client-side paths to `/index.html` to support client-side React routing without 404 errors.
- **Security Headers:** X-Content-Type-Options: nosniff, X-Frame-Options: DENY, and Content-Security-Policy rules enforced at edge nodes.

11.2 Backend Deployment: Render Cloud Web Service

- **Hosting Platform:** Render Cloud Services (Linux environment).
- **Live URL:** `https://aerotrack-backend-1tlt.onrender.com/`
- **Service Configuration (`render.yaml`):**

```
services: - type: web name: aerotrack-backend env: python rootDir: backend buildCommand: pip install -r requirements.txt startCommand: uvicorn main:app --host 0.0.0.0 --port $PORT
```

- **Root Manifest Route (`GET /`):** Returns a JSON API manifest containing service identity, online status, `/docs` link, and `/api/health` target, completely decoupled from static frontend asset serving.
- **CORS Policy:** Explicitly restricted to `https://aerotrack-three.vercel.app` and local development origins, rejecting arbitrary cross-origin requests.

12. RESULTS & VALIDATION

To validate the forecasting engine, a chronological walk-forward benchmark was conducted using completed historical observations across three contrasting urban environments: New Delhi (high-pollution, sub-tropical), London (temperate, maritime), and New York (continental, coastal).

12.1 Chronological Evaluation Benchmark

The model was tested against the scientific standard for short-term time series: the **Naive Persistence Baseline** ($\hat{y}_{t+h} = y_t$). Over 92 days of hourly data, the model was fitted on the first 70% of chronological

history, tuned on 15%, and evaluated on the final 15% completely unseen test period across 13 consecutive walk-forward 24-hour forecast origins (312 total predictions per location).

City / Location	Forecast Horizon	XGBoost MAE	Persistence MAE	XGBoost RMSE	Persistence RMSE	Skill Score vs Persistence
New Delhi, India (High PM2.5, Severe Diurnal Variance)	**+1 Hour**	**2.41 µg/m³**	6.75 µg/m³	**3.00 µg/m³**	8.00 µg/m³	**+62.5%**
	+6 Hours	**15.35 µg/m³**	40.39 µg/m³	**18.27 µg/m³**	51.97 µg/m³	**+64.8%**
	+12 Hours	**32.04 µg/m³**	45.28 µg/m³	**39.02 µg/m³**	59.34 µg/m³	**+34.2%**
	+24 Hours	**29.18 µg/m³**	53.41 µg/m³	**33.09 µg/m³**	64.24 µg/m³	**+48.5%**
New Delhi 24h Horizon Average		**23.88 µg/m³**	39.82 µg/m³	**31.76 µg/m³**	53.48 µg/m³	**+40.6%** ($R^2 = 0.715$)
London, United Kingdom (Low PM2.5, Maritime)	**+1 Hour**	**0.58 µg/m³**	1.22 µg/m³	**0.74 µg/m³**	1.61 µg/m³	**+54.0%**
	+24 Hours	**3.22 µg/m³**	5.14 µg/m³	**4.11 µg/m³**	6.85 µg/m³	**+40.0%** ($R^2 = 0.682$)
New York City, USA (Moderate PM2.5, Coastal)	**+1 Hour**	**0.88 µg/m³**	1.84 µg/m³	**1.12 µg/m³**	2.35 µg/m³	**+52.3%**
	+24 Hours	**4.20 µg/m³**	6.90 µg/m³	**5.34 µg/m³**	8.82 µg/m³	**+39.5%** ($R^2 = 0.694$)

Table 12.1: Multi-Step Evaluation Results vs Naive Persistence Baseline

Key Empirical Findings:

1. **Superiority Over Persistence:** Across all tested geographies, XGBoost significantly outperforms the persistence benchmark. In New Delhi, the overall 24-hour RMSE drops from 53.48 µg/m³ to 31.76 µg/m³, representing a **+40.61% Skill Score improvement**.
2. **Short-Term Precision:** At the +1 hour lead time, the model achieves near-sensor precision: 2.41 µg/m³ MAE in New Delhi, 0.58 µg/m³ in London, and 0.88 µg/m³ in New York.
3. **Diurnal Peak Capture:** While persistence completely misses cyclical morning and evening traffic pollution spikes, the model's cyclical trigonometric encodings ($\sin/\cos$) accurately anticipate rush-hour particulate accumulation.

12.2 Conformal Prediction Empirical Coverage Verification

The Split Conformal Prediction calibration was evaluated to verify whether the 90% theoretical coverage target ($lpha = 0.10$) holds across the unseen test partition:

Evaluation Region	Nominal Coverage Target	Empirical Coverage Achieved	Mean Interval Half-Width (q_{24})	Calibration Strategy
New Delhi, India	90.0%	**90.4%**	±49.65 µg/m³	Distribution-free, 30 holdout origins
London, UK	90.0%	**91.2%**	±5.82 µg/m³	Distribution-free, 30 holdout origins
New York City, USA	90.0%	**90.8%**	±7.45 µg/m³	Distribution-free, 30 holdout origins

Table 12.2: Empirical Coverage Validation for 90% Split Conformal Bands

The empirical coverage closely aligns with the 90.0% nominal target (90.4% in New Delhi, 91.2% in London, 90.8% in New York). This validates that the conformal calibration routine does not suffer from undercoverage or overconservatism, providing reliable bounds for public health safety.

12.3 Automated Test Suite Execution

A comprehensive automated test suite consisting of **51 backend test cases** (via `pytest`) and **5 frontend test cases** (via `vitest`) validates system integrity:

- `backend/tests/test_api_endpoints.py`: Validates CORS headers, EPA PM2.5 piecewise interpolation formulas, upstream null fallbacks, input bounds (-90° to +90° latitude, -180° to +180° longitude), and reverse geocode resilience (14 passed).
- `backend/tests/test_forecast_pipeline.py`: Verifies data leakage prevention, strict causal imputation, opt-in demo flag handling, and upstream error discrimination (11 passed).
- `backend/tests/test_intervals.py`: Verifies monotonic quantile expansion ($q_{t+1} \ge q_t$), calibration caching with data versioning, coordinate mutex locking, and pipeline execution timings (13 passed).
- `backend/tests/test_evaluation_pipeline.py`: Verifies chronological test set partitioning and walk-forward backtesting mechanics (9 passed).
- `backend/tests/test_trends_pipeline.py`: Confirms that 24-hour summary statistics strictly exclude future forecast hours (4 passed).
- `frontend/src/components/__tests__/TrendChart.test.ts`: Verifies SVG reference line rendering, EPA 35.5 µg/m³ boundaries, and data sanitization (5 passed).

```
====================== 51 passed, 1 warning in 14.62s ======================
Test Files 1 passed (1) Tests 5 passed (5) Duration 830ms
```

13. LIMITATIONS

While AeroTrack represents a resilient production system, several scientific and infrastructure limitations must be acknowledged:

1. **Reanalysis Model Latency:** In rural or developing regions without dense physical monitoring networks, Open-Meteo relies on atmospheric chemical transport reanalysis models (e.g., CAMS). These models assimilate satellite observations and global emission inventories, which may exhibit systematic bias or temporal smoothing during localized biomass burning or unpaved dust events.
2. **Single-Step Recursive Compounding Error:** The model generates 24-hour forecasts via recursive autoregressive rollout. While conformal calibration accounts for expanding variance, early-step prediction errors ($\hat{y}_{t+1}$) can compound into later horizons ($\hat{y}_{t+24}$), especially during abrupt weather fronts.
3. **Free-Tier Cloud Sleep Cycles:** Render's free tier spins down backend web services after 15 minutes of inactivity. The initial cold-start request can take 30–50 seconds to boot the container, although subsequent requests resolve in sub-200ms.
4. **Absence of Dynamic Satellite Imagery:** The platform currently forecasts point coordinates without ingesting real-time multispectral satellite raster tiles (such as Sentinel-5P TROPOMI NO2 and aerosol optical depth columns).

14. FUTURE SCOPE

Planned engineering extensions for AeroTrack encompass:

1. **Direct Multi-Horizon Forecasting:** Transitioning from recursive autoregressive rollouts to a direct Multi-Output Gradient Boosting architecture or a Temporal Fusion Transformer (TFT) to eliminate recursive error propagation.
2. **Spatial Kriging & Heatmap Generation:** Implementing client-side WebGL geospatial interpolation (e.g., inverse distance weighting or Gaussian process regression) to render continuous, high-resolution pollution heatmaps across entire metropolitan regions.
3. **Personalized Health Exposure Analytics:** Allowing users to record daily outdoor routes and computing cumulative inhaled PM2.5 dosage based on real-time and forecasted exposure levels.
4. **Push Notification Alert Webhooks:** Providing a notification service allowing users to subscribe to specific coordinates and receive alerts when 24-hour forecasts exceed sensitive health thresholds.
5. **Integration of Low-Cost IoT Sensor Networks:** Ingesting crowd-sourced IoT sensor streams (e.g., PurpleAir, Sensor.Community) with automated calibration algorithms to reconcile local micro-climate readings with regional reanalysis grids.

15. CONCLUSION

AeroTrack successfully demonstrates that high-performance environmental intelligence and machine learning forecasting can be reliably engineered using modern open-source architectures. By combining gradient boosted decision trees with strict temporal causality, distribution-free Split Conformal Prediction, and non-blocking asynchronous Python concurrency, the system bridges the gap between raw atmospheric reanalysis data and accessible, transparent public health tools. The platform achieves a **+40.6% skill score improvement** over standard persistence benchmarks while providing mathematically certified 90% uncertainty intervals that accurately reflect future volatility. By separating static edge presentation on Vercel from scalable backend compute on Render, AeroTrack provides a responsive, production-ready solution capable of empowering citizens, researchers, and policymakers with actionable foresight into the air they breathe.

16. REFERENCES

1. **United States Environmental Protection Agency (US EPA)** (2024). *Reconsideration of the National Ambient Air Quality Standards for Particulate Matter*. Federal Register, 89 FR 16202.
2. **World Health Organization (WHO)** (2021). *WHO Global Air Quality Guidelines: Particulate Matter (PM2.5 and PM10), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide*. Geneva: World Health Organization.
3. **Chen, T., & Guestrin, C.** (2016). *XGBoost: A Scalable Tree Boosting System*. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785–794).
4. **Vovk, V., Gammerman, A., & Shafer, G.** (2005). *Algorithmic Learning in a Random World*. Springer Science & Business Media.
5. **Angelopoulos, A. N., & Bates, S.** (2021). *A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification*. arXiv preprint arXiv:2107.07511.
6. **Open-Meteo** (2024). *Air Quality & Weather API Documentation*. European Centre for Medium-Range Weather Forecasts (ECMWF) & Copernicus Atmosphere Monitoring Service (CAMS). Available: `https://open-meteo.com/`.
7. **OpenStreetMap Contributors** (2024). *Nominatim Geocoding and Reverse Geocoding API*. Available: `https://nominatim.org/`.
8. **Ramírez, T.** (2024). *FastAPI: High Performance Web Framework for Python 3.8+*. Available: `https://fastapi.tiangolo.com/`.